AssetTrust v0.49

Tenant isolation security checkpoint | 11 November 2026 | Internal engineering assessment

RELEASE DECISION: NO-GO - do not process real customer records until the live security gates pass.

What changed

Version 0.49 extends PostgreSQL row-level security (RLS) to the **assets** and **verification_cases** tables. It adds a forward-only migration, scoped database reads and transactions, a production startup privilege check, and a test harness for two distinct organizations.

Existing policies on three private PDF-layout tables and the case_evidence table remain in place. Six tables are therefore configured for RLS **in the source code**. This is not proof that the migration has been applied or that the live database enforces the policies.

Scope of new protections

Control	Implementation
Assets and verification cases	FORCE RLS; tenant-filtered SELECT/UPDATE/DELETE and INSERT checks
Application database access	Validated transaction-local tenant ID; parameterized PostgreSQL statement
Private operations	Assets, cases, dashboards, documents, access, geospatial, reporting and sharing paths updated
Public QR verification	Tenant resolved from existing opaque issued-report reference, not visitor input
Runtime privileges	Production startup denies superuser, BYPASSRLS and protected-table owner

What this does not cover

The users, documents, membership, payments, sessions, audit and other confidential tables are not covered by this new migration. RLS is an additional database barrier, not a replacement for permission checks, least-privilege credentials, parameterized SQL or correct infrastructure configuration.

Verification and evidence

Only the results below were executed in this environment. No claims are made about live infrastructure.

Check	Result
New v0.49 offline tests	9 / 9 passed
Version regression suites v0.03-v0.49	47 / 47 passed
Schema migration checksums	24 / 24 matched
TypeScript / TSX syntax transpilation	102 files / 0 diagnostics
Architecture and CI configuration checks	Passed (static checks)
High-confidence source secret scan	1,923 files / 0 matches; limited patterns
Full application build	FAILED - dependencies not installed
Dependency lockfile	MISSING - reproducible build not established
Live v0.49 PostgreSQL contract	NOT RUN - disposable database unavailable
Deployment release preflight	BLOCKED - 3 of 11 prerequisites present

Remaining release blockers

• No genuine npm lockfile or installed NestJS/React/PostgreSQL packages. The full build did not succeed.

• Migration 024 has not run against a real disposable PostgreSQL instance; cross-organization RLS enforcement and connection-pool reuse remain unverified.

• RLS coverage is not yet database-wide. Existing application role permissions and tenant filters must remain active.

• No independent proof of TLS/firewall/SSH configuration, real browser journeys, malware scanning, dependency remediation, key custody, backup restores or adversarial testing.

• The full canonical Git history was not present in the checkpoint archive. Snapshot scanning cannot establish that historical secrets were never exposed.

Note on test scope

Historical test doubles were updated to accept transaction-local tenant context. Their authorization assertions were retained. Unit tests, static checking and ZIP verification cannot substitute for real PostgreSQL integration or production pen testing.

Required release actions

Execute the next steps on a properly isolated Kali Linux test setup; do not run mutating tests against production.

1. Protect existing data

Perform and verify a restorable PostgreSQL backup and retain encryption and signing keys securely. Audit existing tenant relationships and migration ledger.

2. Establish reproducible builds

Generate and review a genuine package-lock.json, commit it, run npm ci, npm audit, npm test and full API/web builds. Remediate high and critical findings.

3. Prove the database policies

Apply migration 024 with a migration-owner role to a disposable loopback *_ci or *_test database. Run the new two-tenant contract and all earlier PostgreSQL contracts.

4. Validate runtime privileges

Use a non-owner, non-superuser, non-BYPASSRLS application role with minimal grants. Verify fail-closed startup and tenant-scope cleanup after commit, rollback and pooled connection reuse.

5. Execute real workflows

Test two organizations and multiple roles through login, MFA, assets, cases, evidence, documents, reports, recipient sharing and public verification. Include cross-tenant attacks and two API instances.

6. Clear remaining security gates

Verify HTTPS, secure cookies/CSRF, server-side validation and pricing, SSH/firewall hardening, malware scanning, audit logs, Git-history secret scan and credential rotation.

7. Approve a controlled pilot

Retest all fixes and require evidence-backed review with no unresolved critical/high release findings. Document limitations and data-protection responsibilities.

Final verdict: AssetTrust v0.49 is an offline-tested security development checkpoint. Production approval remains **NO-GO**.